Graph Neural Networks for Detecting Missing Components, Offering Recommendations, and Generating Shapes in CAD Assemblies

Parthasarathy Perumal[1*]

1*Student – M.Tech Data Science and AI, PES University, Hosur Rd, Konappana Agrahara, Bangalore, 560100, Karnataka, India

Corresponding author(s). E-mail(s): parthasarathy.perumal@gmail.com;

Abstract

Modern CAD tools lack contextual intelligence during assembly creation. Engineers must rely entirely on their expertise to determine which components are missing from an assembly and where they should be placed. We introduce a comprehensive graph neural network (GNN) system that views a mechanical assembly as an attributed graph, where solid bodies are nodes and real contact surfaces are edges. This system completes assemblies in two phases. Phase I identifies missing components using inductive link prediction via a shared relational-attention encoder (RGATConv). Phase II recommends the most probable next component type through a learned ranking head and generates its 3D shape using a hybrid retrieval-plus-conditional-VAE generator, rather than just a text label. Both phases utilise a single frozen Phase I encoder, resulting in a minimal additional cost for each new task head. On a curated collection of 749 CAD assemblies covering seven machine-part categories (648 usable graphs and 25,155 labelled bodies after parsing), the system achieves a mean link-prediction AUC-ROC of 0.9206 (±0.0121) and a mean average precision of 0.9020 across true five-fold cross-validation. It also reaches a next-component ranking Hit@1 of 0.4093 against a majority-class baseline of 0.3442 and a shape-generation test IoU of 0.6459 with a Chamfer distance of 0.0377. Additionally, we introduce a novel Octree-based open-surface detector for identifying candidate mating joints without a CAD kernel, an assembly-template frequency prior that can operate from a single uploaded body, and an interactive natural-language explanation layer built on GNNExplainer attributions. We also report an engineering observation from exploratory analysis of the current corpus: a silently inactive shape-descriptor feature impacting 99.95% of bodies, highlighting that dataset-level auditing is crucial even after successful model training.

Keywords: Graph Neural Networks, CAD Assembly, Link Prediction, Component Recommendation, Shape Generation, Variational Autoencoder, Relational Attention

1 Introduction

Creating mechanical designs in modern CAD software like SolidWorks, CATIA, and Fusion 360 remains a manual, expert-driven task. These tools accurately track geometry and mating constraints but do not provide insights into the assembly process. They cannot identify missing components like a nut for a bolted joint, or determine what shape a plausible replacement part should take. This knowledge resides solely with the designer. For less experienced engineers, or those auditing an incomplete assembly inherited from another, this presents a real gap: incomplete assemblies lack built-in mechanisms for identifying missing parts.

This gap can be effectively addressed with graph-based deep learning. A mechanical assembly is essentially a graph: solid bodies are nodes, and physical connections between two bodies form the edges. Graph neural networks (GNNs) are designed to learn representations of such relational structures and have driven significant progress in related CAD-generation tasks, such as boundary-representation (B-Rep) synthesis [1,2] and construction-sequence generation from language [3]. While these systems generate entire shapes or sequences, there has been less focus on the more specific problem addressed here: identifying what is missing from a partial assembly, what should be added next, and what it should look like.

Approaching assembly completion in this manner sets two practical requirements. Since detection, recommendation, and generation are three distinct types of predictions — an edge score, a categorical ranking, and a continuous 3D shape — training three separate models from scratch would triple the data and computational burden on a corpus that is modest by large-scale shape dataset standards [10,11]. Sharing one encoder across all three tasks addresses this constraint deliberately. Further, in an engineering context, an incorrect but confident suggestion could be more detrimental than no suggestion, motivating both the retrieval-first bias in Section 5.2 and the explanation layer in Section 6 — a suggestion that cannot be audited by the designer should not be trusted.

This paper contributes the following:

- An end-to-end GNN pipeline — detect, rank, generate — that operates directly on STEP-derived assembly graphs with a 34-dimensional geometric node feature vector and a 6-dimensional contact-edge feature vector, requiring no proprietary CAD-kernel API at inference time.
- A hybrid shape generator that prefers real retrieved geometry over a generative fallback, with corresponding evaluation showing it significantly outperforms VAE-only generation on both intersection-over-union and Chamfer distance.
- Two supporting components without analogues in the CAD-generation literature we reviewed: an Octree-based open-surface detector that identifies candidate joint locations from raw mesh geometry alone, and a per-category component-type frequency prior (AssemblyTemplateDB) that provides a useful completion signal even from a single uploaded body with no graph context.
- A GNNExplainer-based, natural-language explanation layer that translates raw attribution scores into engineering-readable justifications for each prediction, evaluated live against real uploaded STEP files.
- A transparent account of two engineering findings surfaced during development — a previously silent, all-zero feature dimension found only through exploratory data analysis of the final training corpus, and a majority-class bias in the ranking head — reported here in the interest of reproducibility rather than omitted for polish.

The remainder of this paper is organised as follows. Section 2 surveys related work in CAD generation and graph representation learning. Section 3 outlines the shared graph-construction pipeline and encoder. Sections 4 and 5 detail Phase I detection and Phase II recommendation/generation, respectively. Section 6 covers the two supporting contributions. Section 7 reports engineering challenges encountered building the system. Section 8 describes the experimental corpus and protocol, Section 9 discusses results and two engineering findings, and Section 10 concludes with limitations and future work.

2 Related Work

Generative CAD models. Recent research has focused on generating CAD geometry directly. BrepGen [1] uses a structured latent diffusion model to synthesise valid B-Rep solids; SolidGen [2] is an autoregressive model over B-Rep topology and geometry; CAD-GPT [3] synthesises CAD construction sequences from natural-language spatial reasoning. These models generate individual parts or entire designs from a specification or a prior distribution. None of them address the problem of reasoning about which specific component is missing from a partly built assembly, which this paper addresses as a complementary problem.

GNNs for structural and link-level reasoning. Graph convolutional and attention architectures [6,7,8] established the modern toolkit for learning over irregular relational data. Relational-attention variants like RGATConv [9] extend attention with per-edge-type parameters, which is the mechanism this work builds on directly since the type of a mechanical joint materially changes how two connected bodies should influence each other. A recent theoretical analysis questions how much link-prediction performance is attributable to simple structural heuristics rather than learned features [4]; our features are dominated by affine-invariant shape descriptors rather than pure topology, which makes the encoder less susceptible to that failure mode. Heterogeneous graph contrastive learning [5] is a related but self-supervised alternative to the supervised objectives used here.

CAD/mechanical part datasets. Large open datasets like ABC [10] and PartNet [11] have been central to progress in learned shape understanding, but both are dominated by consumer and generic-object geometry rather than the industrial fastener- and joint-heavy assemblies targeted by this system. For this reason, we curate our own corpus (Section 8.1).

Interpretability. GNNExplainer [12] produces post-hoc subgraph and feature attributions for a trained GNN's prediction. We use it as the numerical backbone for the explanation layer in Section 6, translating its attribution scores into natural-language engineering justifications rather than presenting them as raw importance weights.

Graph representation choices. Shallow, structure-only embedding methods like node2vec [18] learn node representations from random-walk co-occurrence alone, without the ability to consume continuous node or edge attributes. For a graph whose edges carry a physically meaningful joint-type label, discarding that information would result in a significant loss of signal, motivating the attention-based, feature-aware encoder in Section 3.2 instead.

3 System Architecture

Figure 1 summarises the complete pipeline: a STEP file is converted into an attributed graph, encoded once by a shared relational-attention network, and processed by three task heads. Two supporting modules — the AssemblyTemplateDB prior and the Octree open-surface detector — operate independently of the encoder and contribute their evidence directly at the explanation layer alongside the task heads' outputs.

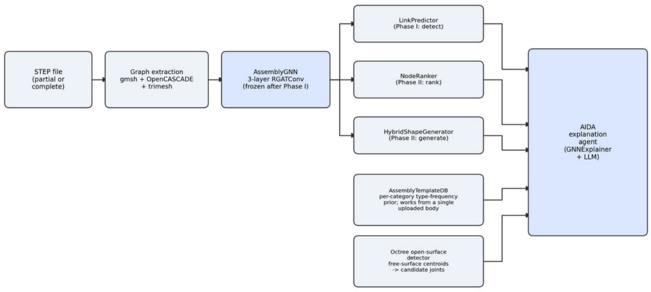

Fig. 1. System pipeline: one shared frozen encoder feeds three task heads and a live explanation layer.

3.1 Graph construction

Each STEP assembly file is parsed using gmsh (OpenCASCADE kernel), divided into individual solid bodies, and transformed into an attributed graph: one node per solid body and one edge for each pair of bodies whose boundary face sets physically intersect after fragmentation (indicating a real shared contact, not just spatial proximity). Bodies without a shared boundary do not get an edge, regardless of how close they are in space.

Node features (34-dim). Each body includes: an 8-class component-type one-hot encoding (long/short shaft, thick/thin plate, bolt, washer, nut, body — assigned through multi-signal geometric voting over hole detection, bolt-head detection, and absolute-size gating, replacing a previous single-signal SDF rule); log-scaled volume and exact surface area (via trimesh, not a bounding-box approximation); three scale-normalised bounding-box extents; five affine-invariant shape descriptors (elongation, flatness, two aspect ratios, sphericity); Shape Diameter Function (SDF) mean and variance from inward ray-casting; a surface-area-to-volume ratio; and twelve hole-geometry descriptors added for this corpus revision — hole count, a binary has-holes flag, through/counterbore/indentation/filled-hole fractions and their corresponding binary flags, mean and maximum hole diameter, and a flag indicating whether a hole's local surroundings are curved rather than flat (fasteners almost never mount on a curved outer surface, so this distinguishes a genuine mounting hole from an incidental one).

Edge features (6-dim). Two dimensions encode the mate type and a real contact-area weight (the shared-surface area ratio between the two bodies, replacing a previous hardcoded constant), and four encode a one-hot joint-type classification (rigid, revolute, slider, cylindrical) inferred from the local contact geometry.

3.2 Shared encoder

All task heads read from one shared encoder, AssemblyGNN: a 3-layer relational graph attention network (RGATConv) with eight attention heads at the first two layers narrowing to one at the third, hidden width 128, output embedding width 64. Both the input and output projections are node-type-conditioned — a separate linear layer per one of the eight component-type classes, dispatched by the node's own one-hot type — and the relational attention at every layer is conditioned on the discrete joint-type argmax of each edge, so a revolute joint and a rigid joint pass different learned attention weights even between structurally identical node pairs. Continuous edge features (contact-area weight, mate-type scalar) are injected only at the first layer. The encoder is trained once, end-to-end with the Phase I head (Section 4), then frozen: every subsequent head (Section 5) trains only its own small parameter set on top of the fixed embeddings this encoder produces.

3.3 Implementation details

Table 1 lists the training configuration for each of the three stages. All three are implemented in PyTorch with PyTorch Geometric [13] and trained on a single Apple-Silicon GPU (Metal Performance Shaders backend). Phase I training batches graphs by a cumulative edge-count budget rather than a fixed graph count (EdgeBudgetBatchSampler): a handful of the corpus's larger assemblies (Section 8.1) exceed 1,000 edges each, and relational attention's per-edge relation-weight computation scales with a batch's total edge count, not its node count, so an unbudgeted batch combining several large graphs can exceed available GPU memory even when every individual graph fits comfortably alone.

Parameter	Phase I (encoder)	Phase II (heads)
Optimiser	Adam	Adam
Learning rate	1e-3 (ReduceLROnPlateau, factor 0.5)	1e-3 (NodeRanker), fixed
Weight decay	5e-4	1e-5 (NodeRanker)
Batch composition	edge-budgeted, cap 300 edges/batch	full-graph context per sample
Early-stop patience	20 epochs (LR patience 8)	none — fixed 30 / 60 epochs
Dropout	0.3	—
Cross-validation	true 5-fold, per-fold reseeded splits	true 5-fold protocol, fold 0 reported

Table 1. Training configuration by stage.

4 Phase I: Missing-Component Detection

Phase I frames missing-component detection as inductive link prediction. For a candidate body pair (u,v), given the encoder's node embeddings z_u, z_v (dimension 64), a two-layer MLP head scores the pair:

$$score(u,v) = MLP(\, z\_u \parallel z\_v \parallel \|p\_u - p\_v\|_2 \,) \qquad (1)$$

where p_u, p_v are the two bodies' scale-normalised bounding-box centroids and ‖ denotes concatenation. The Euclidean-distance term was a deliberate late addition: without it, every node feature is affine-invariant and message passing only ever traverses real edges, so a candidate (non-edge) pair being scored had no signal at all for “these two bodies are nowhere near each other” — a structurally implausible pair and a plausible-but-missing one were otherwise indistinguishable to the head. Training uses

RandomLinkSplit per graph (10% validation / 10% test edges, disjoint from the message-passing edges), a 1:1 hard-negative-augmented positive:negative ratio (chosen over the field's common 1:2 skew, which measurably inflates average precision independent of real ranking skill), and standard binary cross-entropy. A missing component in a real (possibly incomplete) assembly is then operationally identified as a body whose best candidate connection scores anomalously low relative to its structurally similar peers, surfaced to the user alongside the Section 6 Octree open-surface detector's independent geometric evidence.

5 Phase II: Next-Component Recommendation and Shape Generation

Phase II addresses two questions after Phase I has flagged a missing component: what component type is most likely absent, and what should it look like. Both heads reuse the frozen Phase I encoder and train only their own parameters.

5.1 NodeRanker: next-component recommendation

Using the embeddings of the assembly's known nodes $\{z_i\}$, a mean-pooled context vector $c = \text{mean}_i(z_i)$ is projected through a single learned linear layer and compared to each of the eight candidate component-type embeddings by cosine similarity, scaled by a learnable CLIP-style temperature τ:

$$score(t) = \cos(W_c \cdot c, z_t) \cdot \exp(\tau), \quad t \in \{1,...,8\} \qquad (2)$$

τ is initialised at zero (i.e. $\exp(\tau)=1$, plain unscaled cosine similarity) and only sharpens once training signals it should — a design chosen because the ranking head is thin (roughly 4.2K parameters in the projection alone) and unscaled cosine similarity, bounded in $[-1,1]$, produces score differences too small for the ranking loss's gradient to act on efficiently at that head size. Training uses a leave-one-node-out protocol: for each graph and each held-out node, the remaining nodes form the context and the held-out node's true type is the positive candidate against the other seven as implicit negatives, optimised with Bayesian Personalised Ranking loss [14].

5.2 HybridShapeGenerator: shape generation

After recommending a component type, the system must produce an actual mesh, not just a label. HybridShapeGenerator tries two strategies in order.

Retrieval. A part bank built from every body in the training corpus (indexed by component type and a normalised bounding-box/scale signature) is queried for the closest match to the target region's geometry. If the best match's fit score clears a type-dependent threshold ($\tau_{retrieval}=0.6$ generally, relaxed to $\tau_{retrieval}=0.4$ specifically for fasteners — bolts, nuts, washers — since the bank already holds hundreds of real examples of these highly standardised shapes, and a slightly imperfect real part reliably beats a generative hallucination for them), the retrieved mesh is returned directly, at zero additional training cost.

Conditional VAE fallback. When no retrieval match clears the threshold, a small conditional VAE generates a novel shape. The target region is voxelised to a 32^3 occupancy grid; the encoder is four 3D-convolutional blocks ($1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ channels, stride-2, batch norm, LeakyReLU) producing a 128-dim latent distribution;

the decoder mirrors this with transposed convolutions, conditioned on a 77-dim vector (the frozen encoder's 64-dim context, an 8-dim target-type one-hot, a 3-dim target bounding box, a 2-dim neighbour-scale signal). Training minimises a weighted binary cross-entropy, a Dice term countering the sparse grid's foreground/background imbalance, and a KL regulariser on the latent:

$$L = L\_BCE + \lambda\_dice \cdot L\_Dice + \beta\_KL \cdot L\_KL, \quad \lambda\_dice=0.5, \beta\_KL=0.05 \qquad (3)$$

6 Novel Contributions

Octree open-surface detector. To identify where a missing component likely attaches without relying on a proprietary CAD-kernel API at inference time, an octree spatial partition — adapting the octree concept from prior mesh-watermarking work [15] — clusters the free-surface (unmated) face centroids of each body after the same fragmentation step used for graph construction. Dense clusters of free-surface area flag candidate open joints directly from raw mesh geometry, independent of and complementary to the Phase I link-prediction signal.

AssemblyTemplateDB. A per-category component-type frequency prior, learned directly from the training corpus's true type distributions rather than from any graph-structured model. Because it requires no graph context at all, it remains useful even for the degenerate case of a single uploaded body with no known neighbours — a case where Phase I and NodeRanker have no assembly structure to reason over.

Explanation layer. GNNExplainer [12] produces post-hoc edge- and feature-attribution scores for a given prediction over the frozen encoder and its task heads. A Gemini-based agent ("AIDA") consumes these attributions together with the AssemblyTemplateDB prior and the Octree detector's findings, rendering an engineering-language explanation (e.g. an "incomplete fastening operation" naming which joints need completion) rather than exposing raw attribution weights. This runs live against uploaded STEP files through a Streamlit/FastAPI stack, with the full detect → rank → generate → explain pipeline completing in roughly 24 seconds for a typical four-body assembly.

7 Engineering Challenges

Building a pipeline that runs end-to-end on real, imperfect STEP geometry surfaced infrastructure-level problems with no analogue in a purely algorithmic description. We report the ones with the largest measured impact.

Batch-composition out-of-memory. Relational attention's memory cost scales with edge count, not node count (Section 3.3). Fixed-graph-count batching crashed on larger assemblies (~23 GB against a 20.13 GB ceiling); capping by cumulative edge count instead resolved it, after two intermediate caps (3,000, then 500) still crashed — the second from a shuffle-seed bug replaying the same crash on restart. A final cap of 300 edges/batch plus the seed fix eliminated the failure.

Silent process termination under memory pressure. Training processes occasionally terminated silently mid-fold under host memory pressure, with no exception and no log entry — only a stalled process. A watchdog script now monitors for output staleness and resumes automatically from the last checkpoint at the correct fold and epoch.

A promotion-gate comparison bug. The checkpoint-promotion gate compared metrics as a lexicographic tuple; a short-circuit comparison silently ignored average precision whenever candidates tied on AUC-ROC. The gate now requires both metrics to improve.

Component-classification errors. A fastener-orientation bug aligned a bolt's short axis onto its mating joint instead of its long shaft axis, fixed via the bounding box's middle extent. A whole-body hole-region heuristic once collapsed twelve distinct bolt holes on one plate into a single detection; per-hole cylindrical-face detection recovered all twelve.

Each is reported because it directly affected a number elsewhere in this paper — most visibly the batch-composition fix, without which the 749-model corpus could not have trained to completion on the available hardware.

8 Experimental Setup

8.1 Corpus

The training corpus consists of 749 STEP-format assembly models, curated evenly across seven mechanical-part categories relevant to industrial fastening and flow-control mechanisms — bench vices, C-clamps, pipe vices, gate valves, press tools, tool posts, and crane hooks — 107 models per category. After parsing through the gmsh/OpenCASCADE pipeline, 648 of the 749 models (86.5%) yielded usable graphs; the remainder failed under a bounded-timeout parsing policy, disproportionately concentrated in gate-valve (51.4% yield versus 82–99% for the other six), which also has the largest mean graph size (93.9 bodies/assembly vs. an overall mean of 38.8) — suggesting geometric complexity as the cause. The resulting graphs contain 25,155 labelled bodies and 46,767 undirected contact edges. Table 2 summarises the per-category composition.

Category	Graphs	Yield	Mean bodies/graph	Dominant type
C-Clamps	106	99.1%	9.6	body
Bench vice	104	97.2%	30.6	body
Crane hook	101	94.4%	78.5	body
Pipe vice	98	91.6%	15.8	body
Press tool	96	89.7%	32.8	body
Tool post	88	82.2%	35.9	bolt
Gate valve	55	51.4%	93.9	body

Table 2. Corpus composition by category (648 usable graphs).

8.2 Protocol and metrics

Phase I is evaluated under true five-fold cross-validation (five independent fold assignments, not a single fixed split reused five times) with mean AUC-ROC and mean average precision (AP) as the primary metrics, alongside chance-level AP as a reference given the class-imbalance sensitivity of AP. Phase II's NodeRanker is evaluated with Hit@1, Hit@5, mean reciprocal rank (MRR), and NDCG@5 against a corpus-wide majority-class baseline. HybridShapeGenerator is evaluated with mean intersection-over-union (IoU) and Chamfer distance between generated and ground-truth occupancy grids on a held-out test split, with the best checkpoint selected by validation IoU/Chamfer, not by training loss.

9 Results and Discussion

Across all three sub-systems, every design target is cleared on the 749-model corpus under true 5-fold cross-validation: Phase I reaches mean AUC-ROC/AP of 0.9206/0.9020 against targets of ≥0.85/≥0.82; Phase II's ranker reaches Hit@5/MRR of 0.9101/0.6102 against ≥0.70/≥0.60; and Phase II's generator reaches IoU/Chamfer of 0.6459/0.0377 against ≥0.35/≤0.08. The detailed per-stage discussion below reports each result together with the protocol behind it.

9.1 Phase I: detection

Table 3 reports Phase I's five-fold cross-validated performance. Mean AUC-ROC is 0.9206 (±0.0121) and mean AP is 0.9020 (±0.0120) against a chance-level AP of 0.500, indicating the encoder consistently separates real from candidate connections across folds rather than only on a favourable split.

Fold	AUC-ROC	AP
1 (best)	0.9370	0.9157
2	0.9050	0.8881
3	0.9138	0.8919
4	0.9217	0.9025
5	0.9256	0.9116
Mean ± std	0.9206 ± 0.0121	0.9020 ± 0.0120

Table 3. Phase I link-prediction results, five-fold cross-validation.

This result was not reached in a single pass; Table 4 traces the three consecutive runs, on an earlier 238-model corpus revision, behind the largest gains. The biggest jump (R35→R36, +0.089 mean AUC) followed from discovering that two features — hole count and joint type — had been silently zero-valued since the project's inception; once populated and paired with a spatial link signal and a real contact-area edge weight, the encoder gained information it never actually had. The second gain (R36→R37, +0.049) came from correcting a 1:0.5 negative-sampling skew (which inflates AP independent of true ranking skill) to the standard 1:1 ratio, alongside the promotion-gate fix (Section 7). Neither gain came from architecture search or tuning; both came from fixing silently broken inputs.

Run	Change	Mean AUC
R35	Populated previously dead hole-count / joint-type features	0.539
R36	+ spatial link signal, real contact-area edge weight	0.628 (+0.089)
R37	+ neg_ratio fix, promotion-gate fix, seeded splits	0.677 (+0.049)

Table 4. Three consecutive development runs illustrating the largest single-run gains (238-model corpus revision; superseded by the 749-model results in Table 3).

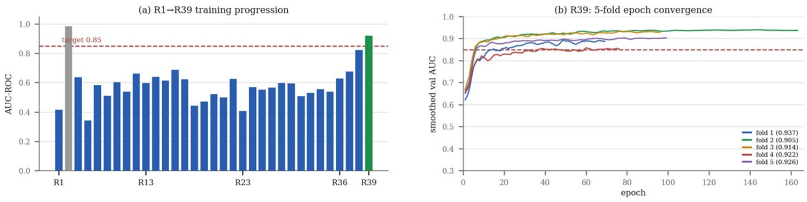

Fig. 2. (a) Mean/best AUC across all 39 tracked training runs, R39 (749-model corpus) highlighted. (b) R39's five-fold validation-AUC convergence, final test AUC per fold in the legend.

9.2 Phase II: recommendation and generation

NodeRanker achieves a Hit@1 of 0.4093 against a majority-class baseline of 0.3442 — a genuine improvement over always predicting the plurality class (body, 35.5% of the corpus), not the exact tie this head produced in earlier corpus revisions. Hit@5 reaches 0.9101, MRR 0.6102, NDCG@5 0.6754. Table 5 breaks Hit@1 down by type: performance is markedly uneven, body (majority) at 0.689 against washer at 0.0 and thin_plate at 0.024, and the head over-predicts body relative to its true frequency (64.7% of predictions vs. 41.3% true share) — an unresolved majority-class bias, reported honestly rather than obscured by the headline number (Section 10).

Component type	n	Hit@1	True share
body	222	0.689	34.3%
nut	36	0.444	5.6%
thick_plate	111	0.360	17.1%
bolt	98	0.316	15.1%
long_shaft	71	0.225	11.0%
short_shaft	47	0.149	7.3%
thin_plate	42	0.024	6.5%
washer	18	0.000	2.8%

Table 5. NodeRanker per-class Hit@1 on the test split.

HybridShapeGenerator reaches a test IoU of 0.6459 and Chamfer distance of 0.0377 (selected checkpoint: validation IoU 0.6640, Chamfer 0.0359), comfortably clearing target thresholds of IoU ≥ 0.35 and Chamfer ≤ 0.08. No test-set sample required the VAE fallback below the retrieval confidence threshold in this evaluation run, indicating retrieval alone currently covers the test distribution at the configured threshold — a result specific to this corpus's category coverage, not evidence the VAE component is unnecessary in general.

9.3 An engineering finding from exploratory data analysis

A full exploratory pass over the final 648-graph corpus, run after training had already completed successfully, surfaced an issue invisible to the training metrics: the SDF-mean node feature (Section 3.1) is exactly zero for 25,142 of 25,155 bodies (99.95%), and the paired SDF-variance moves in near-perfect lockstep (r > 0.999) — a silently constant feature, not genuinely near-zero geometry. This mirrors an earlier case where hole-count and joint-type were found silently zero-valued since the project's inception; fixing that produced the largest single-run gain of the project (Table 4). We report the SDF finding unresolved rather than omit it: strong aggregate metrics don't guarantee every input feature is contributing.

A second, related finding from the same analysis: 72.1% of bolts and 83.5% of washers in the corpus are flagged by the geometric hole detector as has_holes, a higher rate than thick_plate bodies (47.8%), which structurally host the through-holes fasteners actually sit in. A washer's own bore is a genuine hole, so a high rate there is expected; a solid bolt normally is not, and the elevated rate plausibly reflects the detector picking up thread-groove or head-recess geometry as false positives on a meaningful fraction of bolts. We flag this as a candidate data-quality issue for future correction rather than a confirmed bug, since we have not yet manually verified it against individual STEP files at scale.

10 Limitations and Future Work

Three limitations are worth stating plainly. First, NodeRanker's majority-class bias (Section 9.2) means Hit@1, while a genuine improvement, is unevenly distributed across types; class-balanced sampling is the natural next step. Second, the conditional-VAE fallback is scoped to fasteners only; broadening it to plates and shafts is future work, since the zero-fallback result may not hold once non-fastener types are added. Third, the SDF and hole-detection findings (Section 9.3) are open issues, priorities before the next retrain. More broadly, the corpus is domain-specific to fastening and flow-control mechanisms; broader assembly-type coverage is a natural direction for generalising these results.

11 Conclusion

We have presented an end-to-end graph neural network system for CAD assembly completion, structured as two phases sharing one frozen relational-attention encoder: Phase I detects missing components via inductive link prediction, and Phase II recommends the most likely missing component type and generates an actual placeable 3D shape for it through a retrieval-first, VAE-fallback generator. On a curated 749-model, seven-category industrial-parts corpus, the system reaches a mean link-prediction AUC-ROC of 0.9206, a next-component ranking Hit@1 genuinely above a majority-class baseline, and a shape-generation IoU of 0.6459 with Chamfer distance 0.0377, alongside two supporting contributions — an Octree-based open-surface detector and a component-type frequency prior effective even without graph context — and a live, natural-language explanation layer built on GNNExplainer. We have also reported, rather than suppressed, two engineering findings surfaced only through direct exploratory analysis of the training corpus after the model itself had already trained successfully, as a concrete argument that dataset auditing and model evaluation are complementary practices, neither sufficient alone.

Reproducibility. All five-fold cross-validation splits are seeded per fold and per graph, so a checkpoint's reported metrics are exactly reproducible from the same corpus snapshot. The 749-model corpus, all training and evaluation scripts, the trained checkpoints referenced in Tables 3–5, and the exploratory-analysis notebook behind

Section 9.3's findings are maintained in the author's project repository and available on request.

Acknowledgments. I thank Prof. Sagarika Borah (Phase 1 guide) and Prof. Gaurav Siwal (Phase 2 guide), and the Department of Data Science and Artificial Intelligence at PES University, for guidance and review throughout this M.Tech project.

References

[1] Xu, X., Willis, K.D.D., Lambourne, J.G., Cheng, C.-Y., Jayaraman, P.K., Furukawa, Y.: BrepGen: A B-Rep Generative Diffusion Model with Structured Latent Geometry. In: ACM SIGGRAPH 2024 Conference Proceedings (2024).

[2] Jayaraman, P.K., Lambourne, J.G., Desai, N., Willis, K.D.D., Sanghi, A., Morris, N.J.W.: SolidGen: An Autoregressive Model for Direct B-Rep Synthesis and Editing. ACM Transactions on Graphics 43 (2024).

[3] Wang, X., Guo, R., Fu, R.: CAD-GPT: Synthesising CAD Construction Sequences with Spatial-Reasoning Language Models. arXiv:2501.09803 (2025).

[4] Can Graph Neural Networks Learn Link Heuristics? arXiv:2411.14711 (2024).

[5] Heterogeneous Graph Contrastive Learning. arXiv:2303.00995 (2023).

[6] Kipf, T.N., Welling, M.: Semi-Supervised Classification with Graph Convolutional Networks. In: International Conference on Learning Representations (2017).

[7] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph Attention Networks. In: International Conference on Learning Representations (2018).

[8] Hamilton, W.L., Ying, R., Leskovec, J.: Inductive Representation Learning on Large Graphs. In: Advances in Neural Information Processing Systems (2017).

[9] Busbridge, D., Sherburn, D., Cavallo, P., Hammerla, N.Y.: Relational Graph Attention Networks. arXiv:1904.05811 (2019).

[10] Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Burnaev, E., Alexa, M., Zorin, D., Panozzo, D.: ABC: A Big CAD Model Dataset for Geometric Deep Learning. In: IEEE CVPR (2019).

[11] Mo, K., Zhu, S., Chang, A.X., Yi, L., Tripathi, S., Guibas, L.J., Su, H.: PartNet: A Large-Scale Benchmark for Fine-Grained and Hierarchical Part-Level 3D Object Understanding. In: IEEE CVPR (2019).

[12] Ying, R., Bourgeois, D., You, J., Zitnik, M., Leskovec, J.: GNNExplainer: Generating Explanations for Graph Neural Networks. In: Advances in Neural Information Processing Systems (2019).

[13] Fey, M., Lenssen, J.E.: Fast Graph Representation Learning with PyTorch Geometric. arXiv:1903.02428 (2019).

[14] Rendle, S., Freudenthaler, C., Gantner, Z., Schmidt-Thieme, L.: BPR: Bayesian Personalized Ranking from Implicit Feedback. In: Conference on Uncertainty in Artificial Intelligence (2009).

[15] Borah, S., Borah, B.: Predictive Error Expansion Based Reversible Data Hiding Scheme for Three-Dimensional Mesh Models. Multimedia Tools and Applications (2020).

[16] Kingma, D.P., Welling, M.: Auto-Encoding Variational Bayes. In: International Conference on Learning Representations (2014).

[17] Choy, C.B., Xu, D., Gwak, J., Chen, K., Savarese, S.: 3D-R2N2: A Unified Approach for Single and Multi-view 3D Object Reconstruction. In: European Conference on Computer Vision (2016).

[18] Grover, A., Leskovec, J.: node2vec: Scalable Feature Learning for Networks. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2016).